

13th LISA Cosmology Working Group Workshop

June 1–5, 2026 · Institute of Cosmos Sciences (ICC), University of Barcelona

Scientific Software That Lasts:

From Good Practices to AI Tools

Jam Sadiq
IFAE, UAB, Barcelona

Thursday, June 4, 2026

What we are going to discuss?

Motivation

A \$327M spacecraft lost to a unit error — and why the same failure happens in GW science

1

Writing code that lasts

Bad vs. good code, SIGWAY as a model — same physics, very different futures

2

Five habits

Practical disciplines: names, units, tests, documentation, versioning

3

AI tools

Use them well, know their failure modes — and what to watch for

4

September 23, 1999

NASA's Mars Climate Orbiter disappears

\$327M

mission cost

10 mo

journey time

60 sec

to lose it all

Root cause:

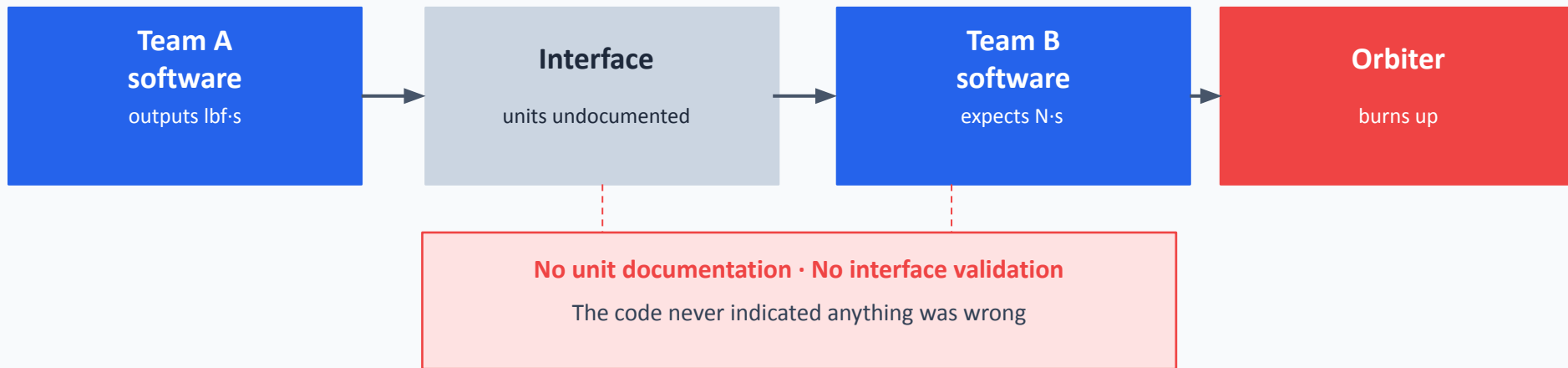
One team output thruster data in pound-force seconds. Navigation software expected newton-seconds.

No unit documentation. No interface validation. The mismatch was completely invisible.

"Bad software doesn't spark. It just gives you the wrong answer — confidently."

A unit mismatch no one could see

The code ran. The tests passed. The spacecraft flew — just not to the right place.



The lesson for science

This wasn't a complex algorithmic failure. It was missing documentation and missing validation. *In science, the exact same failure mode is routine — and the stakes are different only in kind, not in magnitude.*

That was 1999. Aerospace.

Now let's talk about gravitational wave astronomy.

The failure modes are identical.
The consequences are also real.

A silent numerical **error** doesn't lose a spacecraft. It produces a **false detection** — or buries a real one.

BAD CODE

Computing Ω_{GW} — what not to write

This code runs. It produces a plot. But is it correct?

```
import numpy as np
import matplotlib.pyplot as plt

def f(x, y, z):
    a = np.trapz(y * kern(x,z), z)
    return a * 0.8225 * x**2

def kern(k, q):
    u = q / k
    return ((4*u**2 - (1+u**2))**2
            / (1-u**2)**2 * u**2)

k = np.linspace(1e-4, 1, 500)
ps = np.exp(-((k-0.05)/0.01)**2)
out = [f(k0,ps,k) for k0 in k]

plt.plot(k, out)
plt.show()
```

BAD CODE

Computing Ω_{GW} — what not to write

This code runs. It produces a plot. But is it correct?

```
import numpy as np
import matplotlib.pyplot as plt

def f(x, y, z):
    a = np.trapz(y * kern(x,z), z)
    return a * 0.8225 * x**2

def kern(k, q):
    u = q / k
    return ((4*u**2 - (1+u**2))**2
            / (1-u**2)**2 * u**2)

k = np.linspace(1e-4, 1, 500)
ps = np.exp(-(k-0.05)/0.01)**2)
out = [f(k0,ps,k) for k0 in k]

plt.plot(k, out)
plt.show()
```

1

f, kern — names that mean nothing.
What does **f** return physically?

2

0.8225 — magic number. **What equation?**
Which paper? Which limit?

3

linspace — uniform spacing misses the kernel peak. Integral is wrong.

4

No units anywhere. **k** in Mpc^{-1} ? Natural units?
No one can tell.

5

No tests. Will run and plot silently even if the physics is broken.

GOOD CODE

```
# Prefactor: radiation domination
# eq.13, Kohri & Terada 2018
_RD_PREFACTOR = 0.8225

def gw_kernel_rd(k: float,
                 q: np.ndarray
                 ) -> np.ndarray:
    """Kernel  $I^2(k,q)$ , radiation dom.
    eq.(2.14) Espinosa et al. 2018.
    k, q in  $\text{Mpc}^{-1}$ .
    """
    u = q / k
    return ((4*u**2-(1+u**2))**2
            /(1-u**2)**2 * u**2)

def compute_omega_gw(
    k_array: np.ndarray,
    ps: np.ndarray, #  $\text{Mpc}^{-1}$  grid
) -> np.ndarray:
    """Returns  $\Omega_{\text{GW}}$  (dimensionless).
    Log-spaced k_array required.
    """
    ...

# Log-spaced: required for accuracy
k = np.logspace(-4, 0, 500) #  $\text{Mpc}^{-1}$ 
```

Same physics — written to last

Named constant with **equation** reference and **paper citation**

Descriptive name: tells you what it computes physically

Units explicit in docstring — **Mpc^{-1}** , dimensionless, named

Log-spacing documented with justification — not just done silently

Future reader knows exactly which paper to check

Inline unit comment on every array — collaborator-proof

SIGWAY — software built for the future

A LISA Cosmology Working Group collaboration for scalar-induced gravitational waves

Mukhanov-Sasaki solver

Single-field ultra-slow roll inflation → primordial power spectrum P_ζ

Ω_{GW} kernel integration

Second-order GW power spectrum from P_ζ under radiation or matter domination

LISA data pipeline

Foundation for stochastic GW signal analysis for LISA

What SIGWAY does right

- ✓ Tests folder with CI/CD workflow (.github/workflows)
- ✓ Docstrings in every module (even during development)
- ✓ Linked paper: arxiv 2501.11320 — code traceable to physics
- ✓ Open about limitations: 'no stable release yet' — honest documentation
- ✓ requirements.txt + setup.py — reproducible environment
- ✓ MIT/LGPL license — others can build on it legally

Five habits that protect your science

Not for career points. Because your results depend on it.

1 Meaningful names

`compute_omega_gw()` not `f()`. Your future collaborator shouldn't have to reverse-engineer your intent.

2 Document units

Every array. Every function argument. 'k in Mpc^{-1} ' is 5 characters. A unit mismatch can cost a mission.

3 Cite your equations

eq. 2.14 Espinosa 2018. The code is the paper's implementation — make that link explicit.

4 Write tests

Even one test that checks a known limit ($P_{\zeta=0} \rightarrow \Omega_{\text{GW}}=0$) protects against silent regressions.

5 Version and release

Tag your code when you submit the paper. The analysis should be reproducible in 5 years, not just today.

You are writing code for the future.

LISA launches in the 2030s. The software you write today — your parameter estimation pipeline, your waveform models, your injection-recovery tests — will be read, run, and trusted by researchers *who are not yet in the field.*



Readable

Someone else must understand it



Tested

Silent errors are the most dangerous



Citable

Link code to the physics it implements

"Bad code doesn't lose a spacecraft. It produces a false detection — or buries a real one."

Step 1: plan it on paper

Professional software starts with a PLAN.md — not a .py file.

1 Scientific scope — one sentence

What does this package compute? SIGWAY: $\Omega_{\text{GW}}(k)$ from $P_{\zeta}(k)$ under radiation domination. Written down before any code.

2 Inputs, outputs, units — explicit

k in Mpc^{-1} . P_{ζ} dimensionless. Ω_{GW} dimensionless. If units aren't documented before coding, they won't be documented after.

3 Module structure — sketch the tree

kernels.py · spectrum.py · io.py · tests/ · README.md. One file, one responsibility. Decide this before you open an editor.

4 Equations and sources — cite first

Espinosa et al. 2018 eq. 2.14 (kernel). Kohri & Terada 2018 eq. 13 (prefactor). List every non-trivial equation with its paper reference.

5 Write the README before the code

Forces you to state what the package does, how to install it, and how to use it. If you can't write it, you don't understand the package yet.

PLAN.md — SIGWAY example

```
# SIGWAY — PLAN.md

## What this computes
Omega_GW(k) from P_zeta(k),
radiation domination.

## Inputs / Outputs
k_array : np.ndarray # Mpc^-1
ps       : np.ndarray # dimensionless
returns  : Omega_GW   # dimensionless

## Module structure
sigway/
  kernels.py # I^2(k,q)
  spectrum.py # Omega_GW
  io.py       # data loaders
  tests/      # pytest + hypothesis

## Key equations
Kernel: Espinosa+18 eq.2.14
Prefactor: Kohri+18 eq.13

## README written: YES
```

Step 2: implement with discipline

The plan made the structure obvious. Now execute it — with three non-negotiable habits.

1 Names + units in every docstring

```
def gw_kernel_rd(
    k: float,
    q: np.ndarray
) -> np.ndarray:
    """Kernel  $I^2(k,q)$ ,
    radiation domination.
    eq.(2.14) Espinosa+18

    k, q :  $\text{Mpc}^{-1}$ 
    returns: dimensionless
    """
    u = q / k
    return (
        (4*u**2 - (1+u**2))**2
        / (1-u**2)**2 * u**2
    )
```

2 Git as your lab notebook

✗ Bad commit messages

```
git commit -m "update"
git commit -m "fix"
git commit -m "wip"
git commit -m "final"
git commit -m "final2"
```

✓ Good commit messages

```
"add kernel eq.2.14 Espinosa+18"
"add Omega_GW integrator, RD only"
"tag v0.1.0 at paper submission"
```

3 One function, one job

```
# Each function testable
# in isolation

def gw_kernel_rd(k, q):
    # ONLY: compute  $I^2(k,q)$ 
    ...

def compute_omega_gw(
    k_array, ps):
    # ONLY: run the integral
    # calls gw_kernel_rd()
    ...

def load_power_spectrum(
    path):
    # ONLY: read  $P_\text{zeta}$  file
    ...
```

↩ **Callback to habit 5:** git tag v0.1.0 when you submit the paper. The commit history is your audit trail — it shows exactly what code produced which result.

If each function does one thing and is documented, writing tests for it is trivial.

Next: test with Hypothesis →

Beyond example tests — hypothesis for science

Unit tests check one case. Hypothesis checks hundreds — automatically.

Example-based testing (the old way)

```
def test_omega_gw():
    k = np.array([0.01, 0.1])
    ps = np.array([1e-9, 1e-9])
    result = compute_omega_gw(k, ps)
    # Hope these two values
    # cover all edge cases...
    assert result[0] > 0
    assert result[1] > 0
```

Tests 2 hand-picked cases.
Silent for all the edge cases you didn't think of.

Property-based testing (hypothesis)

```
from hypothesis import given
from hypothesis import strategies as st

@given(st.floats(1e-5, 1.0),
       st.floats(0.0, 1e-6))
def test_omega_nonnegative(k, ps_amp):
    k_arr = np.array([k])
    ps = np.full_like(k_arr, ps_amp)
    result = compute_omega_gw(
        k_arr, ps)
    assert result[0] >= 0
```

Hypothesis runs 100+ random (k, ps) pairs.
Finds the smallest failing case automatically.

Properties worth testing in GW code:

$\Omega_{GW} \geq 0$ always · $\Omega_{GW} \rightarrow 0$ when $P_{\zeta} \rightarrow 0$ · result stable under array length changes · units consistent across k conventions

What hypothesis gives you when it finds a bug

Hypothesis finds a failing case → then shrinks it to the simplest possible example

1 found failing input

```
k = [0.00312, 0.47821, 0.99134,  
     0.00041, 0.55902]  
ps = [3.2e-10, 1.1e-9, 8.7e-10,  
      2.2e-10, 5.5e-9]
```

2 shrinking...

```
k = [0.00312, 0.47821]  
ps = [3.2e-10, 1.1e-9]
```

3 minimal counterexample

```
k = [0.0]  
ps = [0.0]  
# k=0 causes division by zero  
# in kern(k, q): u = q/k
```

Terminal output:

```
Falsifying example: test_omega_nonnegative(  
    k=0.0, ps_amp=0.0  
)  
ZeroDivisionError: float division by zero  
    in gw_kernel_rd: u = q / k ← k=0 not guarded
```

You didn't think to test k=0. Hypothesis did. It always will — for every function, every time you change the code.

The world is changing — use it



When calculators arrived in the 1970s, mathematicians didn't stop doing mathematics. They stopped doing arithmetic by hand — and spent their time on harder problems. AI tools are the same thing for scientific software.

Without AI — hours of mechanical work

Writing docstrings with units for every function

Scaffolding a Hypothesis test suite

Setting up README, requirements.txt, setup.py

Writing meaningful git commit messages

Structuring a module tree from a plan

With AI — seconds, then you review

Generates them from your function signature

Writes property tests for each function

Produces it from your existing code

Summarises your diff into a clear message

Proposes kernels.py, spectrum.py, tests/ etc.

Use these tools. But understand what they can and cannot do —
which is exactly what the next slides are about.

next →

A 2024 research pipeline — GitHub Copilot — one default argument

1 Copilot writes the analysis pipeline

A researcher uses GitHub Copilot to build a statistical analysis pipeline in Python. The code looks clean, well-named, and runs without errors. The plots look reasonable. The paper is submitted.

2 A reviewer asks for a rerun

A reviewer requests the analysis be rerun with a slightly different parameter. The researcher goes back to the code — and finds that Copilot had used `np.std()` with the default `ddof=0` (population std) where the context required `ddof=1` (sample std). Every statistical result in the paper was subtly wrong.

3 Caught — but only just

The paper was corrected. But only because someone asked a question. Had the reviewer not asked, it would have remained in the literature. The error produced no warning, no crash, no visible anomaly in the plots.

```
# What Copilot wrote  
std = np.std(samples)           # ddof=0 default
```



```
# What the context required  
std = np.std(samples, ddof=1)  # sample std
```

Using AI tools in scientific code

Claude Code · Cursor · GitHub Copilot · Gemini Code Assist — powerful, but require discipline

1 You own the physics — AI owns the boilerplate

Use AI for file I/O, argument parsing, plotting, test scaffolding. Never let it write the core numerical kernel without you checking every line against the paper.

2 Ask 'why', not just 'what'

Before accepting generated code, ask the tool to explain each step physically. If it can't — or if the explanation doesn't match the paper — the code is wrong.

3 Treat AI output like an unreviewed pull request

It compiled and ran. That is not a review. Read every line. You wouldn't merge code from a new collaborator without reading it.

4 Use AI to write your tests, not to skip them

AI is excellent at imagining edge cases. Ask it to write a Hypothesis suite for your function. Then run it yourself. Best use of AI in scientific code.

5 Document what the AI wrote and why

If a function was AI-assisted, note it in a comment. Future collaborators need to know it wasn't derived by hand from the paper. Context matters for reproducibility.

CLAUDE CODE

What it does

Edits files · runs CLI · uses MCP · creates commits

Tackles narrow tasks (bug fixes) and broad refactors. Works across your whole repo — not just one file.

Full codebase awareness

Searches your project structure automatically. No manual file adding — it finds what it needs.

Tiered permissions

Shows every action before running. You control what it can do autonomously via allowed-tools config.

End-to-end development

Explore · plan · implement · test · commit · debug · refactor · CI/CD. One tool, full lifecycle.

Best practices

1 CLAUDE.md first

Document commands, style, project layout before you start. Claude always reads this. Update it over time — it's your project's memory.

2 Permission discipline

Curate allowed tools. Use auto-accept for safe ops (reads, searches). Use approval mode for writes, commits, and anything irreversible.

3 Context hygiene

/compact between sessions — summarises everything done so far. /clear for a fresh start. Don't let stale context drift your session.

4 You own the review

Claude commits. You read every diff. It ran ≠ it's correct. Same discipline as any other AI tool — maybe more, because it moves faster.

5 Integrate your tools

Install gh CLI, attach MCP servers, put internal tools in CLAUDE.md. Let Claude handle version control mechanics. You handle intent.

"Claude Code is a junior developer who never sleeps, works fast, and needs you to read every pull request."

LIVE DEMO

Let's review SIGWAY with Claude Code

Same codebase. Same principles. Now with an AI pair-programmer — and your critical eye.

What we'll ask Claude Code to do

→ Review codebase for undocumented units and magic numbers

→ Propose a CLAUDE.md for the SIGWAY project

→ Write a Hypothesis test suite for `gw_kernel_rd()`

→ Check git log commit message quality

→ Generate missing requirements.txt and README sections

What to watch for — ask yourself:

? Did it catch the linspace vs logspace issue?

? Did it document the units correctly — or assume?

? Did it read the physics, or just the syntax?

? Would you trust this output without reading it?

? Does the Hypothesis test check real scientific properties?

Opening Claude Code now.

```
cd sigway → claude
```

→ Live demo

Takeaways

things to take

1

The code is the experiment

In modern science your software is not a tool you use to do science — it is the science. A wrong function is a wrong result. Treat it with the same rigour you give your data.

2

Good practices are insurance, not overhead

Documentation, tests, units, version control feel slow when you write them. They feel fast when a collaborator finds your bug in five minutes instead of five months.

3

Be aware of AI issues

AI tools produce plausible-looking code at speed — but they don't know your physics, your units, or your conventions. Silent errors, wrong defaults, misplaced trust. Your critical review is non-negotiable.

Beyond example tests — hypothesis for science

Finding the edge case you never thought to write.

The function under test

```
import numpy as np

def normalize_strain(segment):
    """Scale GW strain to
    zero mean, unit variance."""
    mean = np.mean(segment)
    std = np.std(segment)
    return (segment - mean) / std
```

Edge case

Input:

[3., 3., 3.]

std = 0

÷ 0

nan / inf

No error raised.
Silent pipeline bug.

With Hypothesis – finds it automatically

```
from hypothesis import given
from hypothesis.extra.numpy import arrays
from hypothesis import strategies as st

@given(arrays(dtype=np.float64,
              shape=st.integers(2, 200),
              elements=st.floats(-1e6, 1e6)))
def test_normalize(segment):
    out = normalize_strain(segment)
    assert np.all(np.isfinite(out))

# ✗ Falsifying example found:
#   segment = array([0., 0.]
```

Key insight: You'd never write [0., 0.] as a test input — you're thinking about signals, not edge cases. **Hypothesis isn't thinking about signals at all.**

The xz Utils compromise — and why it matters for AI-assisted science

2021

● A new contributor 'Jia Tan' appears. Helpful, thorough, patient. Builds trust over two years.

2023

● Gains commit access. Introduces subtle, hard-to-read obfuscated code across multiple commits.

Mar 2024

● A Microsoft engineer notices SSH logins are ~500ms slower than expected. Investigates.

Apr 2024

● CVE-2024-3094 published. Backdoor found in xz-utils — shipped in almost every Linux system.

The lesson: plausible-looking code is not the same as correct code. AI makes plausible-looking code cheaper than ever.